Informatik

[redacted]

Schuljahr 2025/2026

# Machine Learning - Grenzen und Möglichkeiten von Deep Reinforcement Learning am Beispiel „Vier Gewinnt“

Jonathan Dersthof

[redacted]

# Inhalt

1 Einleitung ..... 3

2 Theoretische Grundlagen ..... 4

2.1 Machine Learning vs. klassische Programmierung ..... 4

2.2 Arten von Machine Learning ..... 5

2.3 Künstliche Neuronale Netze ..... 5

2.4 Reinforcement Learning ..... 6

3 Deep Q-Networks (DQN) ..... 8

3.1 Von der Q-Tabelle zum neuronalen Netz ..... 8

3.2 Stabilität des Lernprozesses ..... 8

3.3 Double Deep Q-Learning (DDQN) ..... 9

4 Implementierung ..... 9

4.1 Definition der Spielumgebung ..... 9

4.2 Belohnungssystem ..... 10

4.3 Netzwerkaufbau und Hyperparameter ..... 10

4.4 Trainingsstruktur ..... 11

5 Auswertung des Experiments ..... 12

5.1 Auswahl der auffälligen Trainingseinheiten ..... 12

5.2 Vorstellung Trainer 3 ..... 13

5.3 Vergleich zu Trainer 7 ..... 14

5.4 Vergleich zu Trainer 4 ..... 15

5.5 Vergleich zu Trainer 1 ..... 16

5.6 Vergleich Trainer 11 ..... 17

5.7 Zusammenfassung und Sonderheiten ..... 17

6 Fazit ..... 18

7 Literaturverzeichnis ..... 19

# 1 Einleitung

Machine Learning ist in den letzten Jahren von einem Nischenthema der Informatik zu einer Technologie geworden, die zunehmend Einzug im Alltag erhält: von Sprachassistenten über die allseits bekannten Sprach- und anderen generativen Modelle bis hin zu medizinischen Diagnosewerkzeugen ist alles dabei.

Einen besonders faszinierenden Teilbereich stellt dabei das Reinforcement Learning dar, bei dem ein Agent nicht durch vorgegebene Beispiele, sondern ausschließlich durch Interaktion mit seiner Umgebung und dem Sammeln von Erfahrungen lernt.

Die Fähigkeit, komplexe Spiele auf menschlichem oder übermenschlichem Niveau zu meistern, gilt dabei als einer der eindrücklichsten Belege für das Potenzial dieser Methode. Bekannte Beispiele wie DeepMinds AlphaGo oder die DQN-Agenten, die Atari-Spiele beherrschten, haben gezeigt, was Deep Reinforcement Learning leisten kann, nicht immer ohne Probleme.

Die vorliegende Facharbeit untersucht diese Möglichkeiten, aber auch Grenzen am konkreten Beispiel des Brettspiels „Vier Gewinnt“. Vier Gewinnt eignet sich dafür, da es einerseits einfach genug ist, um innerhalb vertretbarer Zeit Modelle zu trainieren, andererseits komplex genug, um taktische Tiefe zu erfordern, anders als Tic-Tac-Toe zum Beispiel. Ziel der Arbeit ist es, einen funktionsfähigen Deep-Q-Network-Agenten zu implementieren, dessen Lernverhalten systematisch zu analysieren und daraus konkrete Aussagen über die Stärken und Schwächen des Ansatzes abzuleiten.

Im Zentrum stehen dabei folgende Fragen: Wie effektiv kann ein DQN-Agent Vier Gewinnt erlernen? Welchen Einfluss haben Hyperparameter wie Netzwerkgröße, Lernstrategie und Belohnungsstruktur auf den Trainingserfolg? Und wo liegen die prinzipiellen Grenzen eines DQN bei einem geometrischen Spiel?

Zu Beginn behandle ich das Konzept von Machine Learning in Abgrenzung zur klassischen deterministischen Programmierung, sowie unterliegende Grundkonzepte. Anschließend erkläre ich den für die Implementierung gewählten Ansatz des Deep Reinforcement Learnings und wie er sich von anderen Machine Learning Ansätzen unterscheidet. Zum Abschluss Werte ich die Trainingsdaten des implementierten Systems aus und beantworte Anhand der Ergebnisse die anfangs gestellten Fragen.

# 2 Theoretische Grundlagen

## 2.1 Machine Learning vs. klassische Programmierung

Der fundamentale Unterschied zwischen klassischer, deterministischer Programmierung und Machine Learning (ML) liegt in der Art und Weise, wie ein System eine Problemlösung generiert. In der traditionellen Softwareentwicklung ist der Programmierer dafür verantwortlich explizite Regeln und logische Abfolgen (Algorithmen) zu definieren. Das System folgt diesen Anweisungen strikt nach dem Prinzip:

*Eingabedaten + Regeln = Ergebnis (EVA-Prinzip)*

Dieser Ansatz stößt jedoch an seine Grenzen, sobald die zugrundeliegenden Muster zu komplex oder zu variabel sind, um sie manuell in Code zu fassen. In der Realität wird wären Anwendungsbeispiele etwa Gesichtserkennung oder Sprache.

Machine Learning kehrt diese Regeln um. Hier werden dem System Daten und oft auch die gewünschten Ergebnisse präsentiert, woraufhin der Algorithmus selbstständig die zugrundeliegenden statistischen Regeln identifiziert:

*Eingabedaten + Ergebnisse = Regeln*

Ein ML-Modell lernt somit aus Erfahrung, um auf neue, unbekannte Daten zu generalisieren, anstatt lediglich vordefinierte Pfade einzuhalten.[1]

[1] *Vgl. Sutton, R. S. / Barto, A. G. (2018): Reinforcement Learning: An Introduction, 2. Aufl., S. 1.*

## 2.2 Arten von Machine Learning

Innerhalb des Machine Learnings lassen sich drei wesentliche Lernansätze unterscheiden:

1. Supervised Learning: Das Modell lernt anhand eines Datensatzes, der bereits mit den korrekten Lösungen (Labels) versehen ist.[2] Es versucht, eine Funktion zu finden, die Eingaben auf die bekannten Zielwerte abbildet.
2. Unsupervised Learning: Hier arbeitet das System mit ungelabelten Daten. Ziel ist es, eigenständig verborgene Strukturen oder Gruppierungen innerhalb der Daten zu finden, ohne dass eine externe Zielvorgabe existiert.[3]
3. Reinforcement Learning (Bestärkendes Lernen): Im Gegensatz zu den vorgenannten Formen lernt ein Agent hier durch die Interaktion mit einer Umgebung. Wie Sutton und Barto (2018, S. 1) ausführen, wird dem Lernenden nicht vorgegeben, welche Aktionen er ausführen soll. Stattdessen muss er durch Versuchen herausfinden, welche Handlungen die höchste Belohnung (Reward) einbringen. Diese Variante sich zeichnet sich durch den Konflikt zwischen der Nutzung bereits bekannter guter Aktionen (Exploitation) und dem Erproben neuer Möglichkeiten (Exploration) aus.[4]

## 2.3 Künstliche Neuronale Netze

Um Reinforcement Learning auf komplexe Probleme, wie die Verarbeitung von Rohpixeln eines Bildschirms, anzuwenden, ist der Einsatz von „Funktionsaproximatoren“ notwendig.[5] In diesem Kontext dienen künstliche neuronale Netze (Artificial Neural Networks, ANN) dazu, die Generalisierung von Erfahrungen zu ermöglichen[6], da eine tabellarische Erfassung aller möglichen Zustände (z. B. jede mögliche Pixelkonstellation) rechnerisch nicht bewältigbar wäre.[7]

Ein künstliches neuronales Netz besteht aus mehreren Schichten miteinander verbundener „Neuronen“. In einem sogenannten Feedforward-Netz fließt die Information

---

[2] *Vgl. ebd., S.2.*
[3] *Vgl. ebd.*
[4] *Vgl. ebd., S.3.*
[5] *Vgl. Mnih, V. et al. (2015): Human-level control through deep reinforcement learning, in: Nature, Vol. 518, S. 529.*
[6] *Vgl. Sutton / Barto (2018), S. 223.*
[7] *Vgl. ebd., S. 197 f.*

ausschließlich in eine Richtung: von der Eingangsschicht über eine oder mehrere verborgene Schichten (Hidden Layers) zur Ausgangsschicht.[8]

Die einzelnen Schichten des Netzes berechnen eine gewichtete Summe ihrer Eingänge und wenden darauf eine nicht-lineare Aktivierungsfunktion an.[9] In modernen Architekturen wie dem DQN findet primär die ReLU-Funktion (Rectified Linear Unit) Anwendung, definiert als:

$$f(x) = max(0, x)$$ [10]

*Also wenn die Eingabe positiv ist, wird sie übernommen, ansonsten wird 0 ausgegeben.*

Diese Nichtlinearität ermöglicht es dem Netz, komplexe Zusammenhänge zwischen der Eingabe und den erwarteten Belohnungen abzubilden.[11]

Der Lernprozess des Netzes erfolgt über den Backpropagation-Algorithmus. Dabei wird der Fehler zwischen der aktuellen Vorhersage des Netzes und einem Zielwert berechnet. Dieser Fehler wird mittels des Gradientenabstiegsverfahrens (Stochastic Gradient Descent) rückwärts durch das Netz geleitet, um die Verbindungen so anzupassen, dass der Vorhersagefehler minimiert wird.[12] Im Falle des DQN wird das Netz so trainiert, dass es sich an die optimale Aktionswertfunktion annähert.[13]

## 2.4 Reinforcement Learning

Das Lernen aus der direkten Interaktion mit der Umgebung wird formal als Markov-Entscheidungsprozess (Markov Decision Process, MDP) beschrieben.

---

[8] *Vgl. ebd., S. 223 f.*
[9] *Vgl. ebd. S. 226.*
[10] *Vgl. Mnih et al. (2015), S. 530.*
[11] *Vgl. Mnih, V. et al. (2013): Playing Atari with Deep Reinforcement Learning, S. 5.*
[12] *Vgl. Sutton / Barto (2018), S. 226 f.*
[13] *Vgl. van Hasselt, H. et al. (2015): Deep Reinforcement Learning with Double Q-learning, S. 2.*

Ein MDP besteht aus:

- States (s): den Zuständen
- Actions (a): den Handlungsmöglichkeiten
- Rewards (r): der Rückmeldung

Die wichtigste Regel hierbei ist die Markov-Eigenschaft, welche besagt, dass für die Entscheidung nur der jetzige Zustand wichtig ist. Die gesamte Vergangenheit des Spiels spielt für den nächsten Zug keine Rolle.[14] Das Ziel des Agenten ist es, eine Strategie (Policy) zu entwickeln, die langfristig die höchste Summe an Belohnungen einsammelt.[15]

Um die optimale Strategie zu finden, nutzt man die Aktionswertfunktion *Q(s, a).* Sie gibt den erwarteten Ertrag an, wenn in einem Zustand eine Aktion gewählt wird. Der Zusammenhang zwischen dem aktuellen Wert und dem Wert des Folgezustands wird durch die Bellman-Gleichung definiert:

$$Q*(s,a) \;=\; E\left[R \;+\backslash\, gamma \;*\max_{a'} Q*(s',a')\right]$$

Dies bedeutet: Der Wert einer Aktion ist die Summe aus der sofortigen Belohnung und dem bestmöglichen zukünftigen Wert.[16]

Der Q-Learning-Algorithmus ermöglicht es dem Agenten, diese Werte iterativ zu verbessern. Dabei wird nach jeder Aktion die Differenz zwischen der aktuellen Schätzung und dem tatsächlich beobachteten Ertrag berechnet (TD-Fehler). Der Q-Wert wird dann in Richtung dieses Fehlers angepasst.

Dabei bestimmt die Lernrate, wie stark neue Informationen das bisherige Wissen überschreiben.[17] Da beim Q-Learning die Schätzung des optimalen Wertes unabhängig von der aktuell ausgeführten Strategie erfolgt, spricht man von einem Off-Policy-Verfahren.[18]

---

[14] *Vgl. Sutton / Barto (2018), S. 49.*
[15] *Vgl. Mnih et al. (2015), S. 529.*
[16] *Vgl. Mnih et al. (2013), S. 3.*
[17] *Vgl. Sutton / Barto (2018), S. 131.*
[18] *Vgl. ebd.*

# 3 Deep Q-Networks (DQN)

## 3.1 Von der Q-Tabelle zum neuronalen Netz

Im klassischen Q-Learning werden die gelernten Werte in einer Tabelle gespeichert. Für einfache Probleme funktioniert dies gut, doch bei komplexeren Problemen stößt dieser Ansatz an seine Grenzen. Obwohl das Spielbrett mit 6x7 Feldern überschaubar wirkt, gibt es theoretisch Billionen von möglichen Spielzustände. Eine Tabelle, die für jeden dieser Zustände einen Wert speichert, würde den Arbeitsspeicher der meisten Computer überfordern und somit das Training extrem verlangsamen.[19]

Die Lösung besteht darin Q-Learning mit künstlichen neuronalen Netzen zusammenzuführen. Anstatt den exakten Wert nachzuschlagen, gibt man dem Netz den aktuellen Zustand (state) als Eingabe. Das Netz berechnet daraufhin eine Schätzung der Q-Werte für alle möglichen Aktionen.[20]

## 3.2 Stabilität des Lernprozesses

Das Training eines neuronalen Netzes im Reinforcement Learning ist mathematisch instabil. Zwei Techniken sind entscheidend, um dieses Problem zu lösen und einen stabilen Agenten für „Vier Gewinnt“ zu trainieren:

1. Experience Replay (Erfahrungswiederholung): In einem Spiel folgen Zustände direkt aufeinander und hängen stark voneinander ab. Würde das Netz direkt aus diesen aufeinanderfolgenden Zügen lernen, würde es „vergessen“, was es in anderen Situationen gelernt hat.[21] Beim Experience Replay speichert der Agent seine Erfahrungen (Zustand, Aktion, Belohnung, nächster Zustand, Beendet) in einem Speicher (Replay Buffer/Memory). Für das Training werden dann zufällige Stichproben (Batches) aus diesem Speicher ausgewählt. Dies bricht den Zusammenhang zwischen den aufeinanderfolgenden Schritten auf und führt zu einem effizienteren Lernprozess.[22]

---

[19] *Vgl. Mnih et al. (2013), S.1.*
[20] *Vgl. Mnih et al. (2015), S.529..*
[21] *Vgl. ebd., S.530.*
[22] *Vgl. Mnih et al. (2013), S. 5.*

2. Target Networks (Ziel-Netzwerke): Beim Q-Learning verändert sich das Ziel (Target), auf das das Netz trainiert wird, mit jedem Update, da das Netz seine eigenen Vorhersagen zur Berechnung der Zielwerte nutzt.[23] Um dies zu verhindern, nutzt man beim DQN ein zweites, identisches Netz (das Target Network), das nur alle paar hundert Schritte aktualisiert wird und währenddessen stabil bleibt.[24]

## 3.3 Double Deep Q-Learning (DDQN)

Ein bekanntes Problem des Standard-DQN ist die Tendenz, Q-Werte systematisch zu überschätzen. Da der Algorithmus immer die Aktion mit dem Maximum der geschätzten zukünftigen Werte wählt, werden Fehler, bei denen das Netz den Wert zu hoch einschätzt, verstärkt.[25] Das Double DQN löst dies, indem es die Auswahl der besten Aktion und die Bewertung dieser Aktion voneinander trennt. Das eine Netz entscheidet, welche Aktion die beste ist, während das Target Network den Wert dieser Aktion berechnet. Dies führt nachweislich zu realistischeren Schätzungen und einer besseren Spielstärke.[26]

# 4 Implementierung

Nachdem ich die theoretischen Grundlagen vorgetragen habe, stelle ich nun meine Umsetzung der genannten Konzepte am Beispiel eines Vier Gewinnt Agenten, mithilfe der Programmiersprache Python und des Frameworks PyTorch, welches Machine Learning Funktionen bereitstellt.

## 4.1 Definition der Spielumgebung

Das Spielfeld besteht klassisch aus 7 Spalten und 6 Reihen. Dies wird im Programm als Matrix und der Zustand jedes einzelnen Feldes entweder als 0 (frei), 1 (vom Agenten besetzt) oder -1 (vom, aus der Sicht des Agenten, Gegner besetzt) dargestellt. Dieser Zustand wird als eindimensionaler Vektor dem Agenten übergeben.

---

[23] *Vgl. Mnih et al. (2015), S. 530.*
[24] *Vgl. ebd.*
[25] *Vgl. van Hasselt et al. (2015) S.2.*
[26] *Vgl. ebd., S.2.*

In jedem Zug stehen dem Agenten maximal 7 Aktionen zur Verfügung, die den 7 Spalten entsprechen. Sollte eine Spalte voll sein, wird sie durch den potentiellen Q-Wert = -∞ maskiert, wodurch der Agent in keinem Fall eine illegale Aktion machen kann. Dadurch soll das Training beschleunigt werden.

## 4.2 Belohnungssystem

Die Wahl der Belohnungsfunktion ist entscheidend für den Lernerfolg. Ein zu komplexes System kann den Agenten verwirren, während ein zu einfaches System („Sparse Reward“) das Lernen stark verlangsamt, da der Agent erst nach vielen zufälligen Zügen eine Rückmeldung erhält.[27] In dieser Arbeit wurde folgendes System implementiert:

- Sieg: +1,0
- Unentschieden: +0,1
- Niederlage: -1,0
- Bloßes Überleben (nicht verlieren) eines Zuges: +0,001 oder -0,005

Durch die Wahl dieser Werte sind die meisten Szenarien abgedeckt und ein Agent sollte einerseits lernen zu gewinnen, vermeiden zu verlieren und je nach Belohnung für das Überleben auf eine eher defensive Spielweise zu setzen (langanhaltende Spiele belohnt) oder schnellstmögliches Gewinnen anzustreben (Belohnung höher je weniger Züge)

## 4.3 Netzwerkaufbau und Hyperparameter

Das Netz besteht immer aus einer Eingangsschicht mit 42 (6 x 7) Neuronen, drei versteckten Schichten derselben Größe (im auszuwertenden Training je 128 oder 256 Neuronen) und einer Ausgangsschicht mit 7 Neuronen (Die Aktionen). Auf die versteckten Schichten wird die vorher genannte ReLU-Funktion angewendet.

Für den Lernprozess wurden außerdem folgende Parameter gesetzt:

- Die Lernrate beträgt immer 0,001
- Epsilon startet immer bei 1,0, wodurch der Agent immer zu Beginn ausschließlich zufällig agiert und somit die Umgebung erkundet (Exploration). Dieser Wert sinkt im Verlauf einer Trainingseinheit auf 0,1, wodurch der Agent zu 99% das eigene „Wissen“ anwendet (Exploitation) und nur zu 1% erkundet.

[27] *Vgl. Sutton/ Barto (2018), S. 53 f.*

## 4.4 Trainingsstruktur

Die von mir aufgesetzte Trainingsstruktur hat drei Hauptstrategien:

1. Basis-Training: das Basistraining ist die simpelste Form des Trainings. Hier trainiert ein Agent gegen einen Zufallsgegner. Epsilon startet immer bei 1,0 und sinkt auf 0,01.
2. Self-Play: dieses Training ist eine deutlich interessantere Variante. Hier trainieren zwei Agenten gegeneinander. Je nachdem kann nur einer der beiden Agenten lernen und der andere wendet nur das Wissen an oder beide lernen, wodurch das Training jedoch unter Umständen sehr instabil werden kann, da beide Agenten versuchen sich an ihren jeweiligen Gegner anzupassen, dieser im nächsten Spiel jedoch auch angepasst ist und somit schwer vorherzusehen für den Agenten ist.
3. League-Play: Mit dieser Trainingsvariante sollte vermieden, dass sich der Agent zu sehr an einen festen Gegner anpasst (Overfitting). Statt immer gegen den gleichen Agenten zu spielen, spielt der Agent hier gegen zufällig gewählte Agenten aus einem Agenten-Pool oder gegen einen Zufallsgegner.
4. Volles Training: Das volle Training ist eine Kombination aus Basis-Training und mehreren Iterationen von League-Play. Am Ende jeder Trainingseinheit wird der finale Agent dem Agenten-Pool angehangen. Somit wächst über die Iterationen der Agenten-Pool und auch die Länge des Trainings, um die Menge an unterschiedlichen Gegnern auszugleichen.

Um verschiedene Ansätze miteinander zu vergleichen, sind im Rahmen der Arbeit Agenten mit 16 verschiedene Kombinationen an Parametern trainiert worden. Der grundlegende Versuchsaufbau war immer identisch: Ein volles Training bestehend aus einem 100.000 Spielen langem Basis-Training und 20 Iterationen von League-Play (Länge startet bei 10.000 Spielen und wächst mit jeder Iteration um 10.000 bis schlussendlich die Länge der finalen Trainingseinheit 200.000 Spiele beträgt).

Die beigelegten Grafiken stellen den Loss, also die Differenz zwischen dem vom Netz vorhergesagten Q-Wert und dem durch die Bellman-Gleichung berechneten Zielwert aus sofortiger Belohnung und bestem geschätzten Folgezustand pro Spiel, die Siegesquote gegen einen Zufallsgegner (alle 100 Spiele wird der Trainingsprozess gestoppt und der derzeitige Agent simuliert 100 Spiele gegen einen Zufallsgegner) und schlussendlich die gesamte über ein Spiel akkumulierte Belohnung des Agenten.

# 5 Auswertung des Experiments

Die im Rahmen dieser Arbeit erhobenen Trainingsdaten umfassen 16 vollständige Trainingsläufe mit insgesamt 35.200.000 simulierten Spielen. Im Folgenden werden zunächst die für die Auswertung relevanten „Trainer“ ausgewählt und begründet. Anschließend werden die Ergebnisse mit Bezug auf die drei in der Einleitung aufgeworfenen Fragen ausgewertet werden: Wie effektiv kann ein DDQN-Agent Vier Gewinnt lernen, welchen Einfluss haben Hyperparameter auf den Trainingserfolg und wo liegen die Grenzen des Ansatzes?

## 5.1 Auswahl der auffälligen Trainingseinheiten

Da eine vollständige Besprechung aller 16 Trainer den Rahmen dieser Arbeit sprengen würden, wurden fünf Trainer nach zwei Kriterien ausgewählt: Erstens, ermöglichen sie kontrollierte Paarvergleiche (Trainer, die sich in genau einem Parameter unterscheiden) und Aussagen über den Einfluss einzelner Hyperparameter.

Als Referenztrainer dient Trainer 3, da er das stabilste und konsistenteste Trainingsverhalten aller 16 Trainer, bezogen auf sowohl Loss als auch Win-Rate und Reward-Werte, zeigt. Er bildet die Grundlage, gegen die alle anderen Trainer verglichen werden.

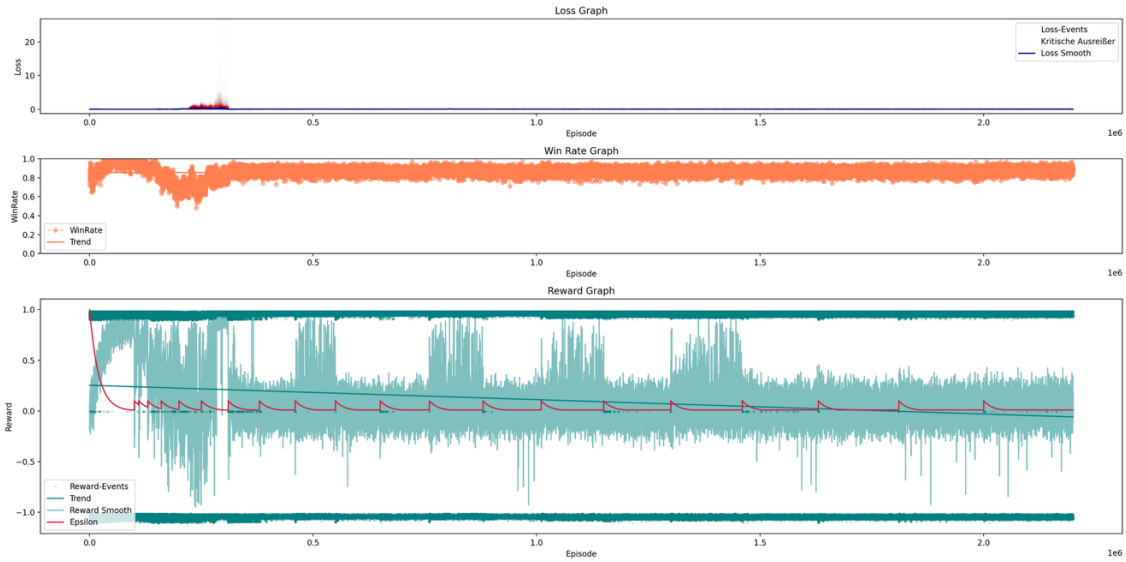
Für den kontrollierten Vergleich wurden Trainer 7 (identisch zu Trainer 3, jedoch Gamma = 0,99 statt 0,95), Trainer 4 (identisch zu Trainer 3, jedoch survive_reward = +0,001 statt -0,005), Trainer 1 (identisch zu Trainer 3, jedoch niedrigere batch_size), sowie Trainer 11 (identisch zu Trainer 3, jedoch hidden_size = 128) gewählt, da sie sich von Trainer 3 jeweils in exakt einem Parameter unterscheiden.

Die übrigen Trainer zeigen zum Großteil keine stark abweichenden Muster und bestätigen die nachfolgend beschriebenen Tendenzen ohne neue Informationenn hinzuzufügen.

## 5.2 Vorstellung Trainer 3

Zu Beginn stelle ich die Hyperparameter von Trainer 3 dar:

- Versteckte Schichtengröße: 256 Neuronen
- Gamma: 0,95
- Paketgröße: 128
- Überlebensbelohnung: -0,005




In Trainer 3 verläuft das Training abgesehen von einem kurzen Einbruch der Win-Rate und einem Anstieg des Loss in der Übergangsphase von Basis-Training zu League-Play sehr stabil. Vor allem zum Ende des Trainings hin erkennt man eindeutig, wie sich das Niveau der Win-Rate, der Loss und sogar die durchschnittliche Belohnung stabilisiert.

Durch die Kombination aus niedrigerem Gamma Wert und negativer Belohnung setzt der Agent auf ein kürzeres Spiel und einen schnellen Sieg. Diese Strategie scheint sich auszuzahlen.

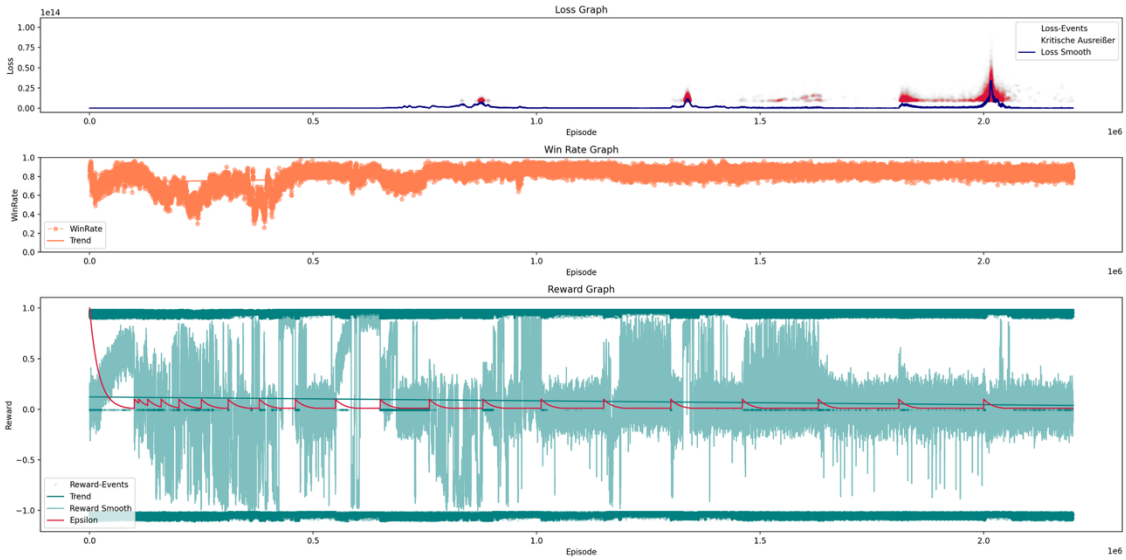
In exemplarischen Testspielen gegen einen Menschen zeigt sich, jedoch: Der Agent scheint zu sehr an die eigenen Erfahrungen angepasst zu sein. Die späteren Modelle (spätestens ab League-Play 10) verfolgen immer dieselbe Strategie den Stein in die gleiche Spalte zu legen, bis diese voll ist. Das scheint gegen einen Zufallsgegner sehr effektiv, daher die hohe Win-Rate. Kontraintuitiv sind die Modelle mit dem höchsten Loss (vor allem League-Play-Model 6) im Spiel gegen einen Menschen deutlich dynamischer (sie blockieren) und somit deutlich erfolgreicher.

Dieses Phänomen ist in der Literatur als Overfitting bekannt und tritt hier in einer für Reinforcement Learning spezifischen Form auf: Der Agent überpasst sich nicht an einen festen Datensatz, sondern an das Verhaltensmuster der Gegner im Agentenpool. Da dieser Pool aus Instanzen desselben Agenten besteht, fokussiert sich das Training um eine enge Klasse von Strategien und entwickelt blinde Flecken für alles, was außerhalb dieser Klasse liegt.

## 5.3 Vergleich zu Trainer 7

Die Hyperparameter von Trainer 7:

- Versteckte Schichtengröße: 256 Neuronen
- Gamma: 0,99
- Paketgröße: 128
- Überlebensbelohnung: -0,005



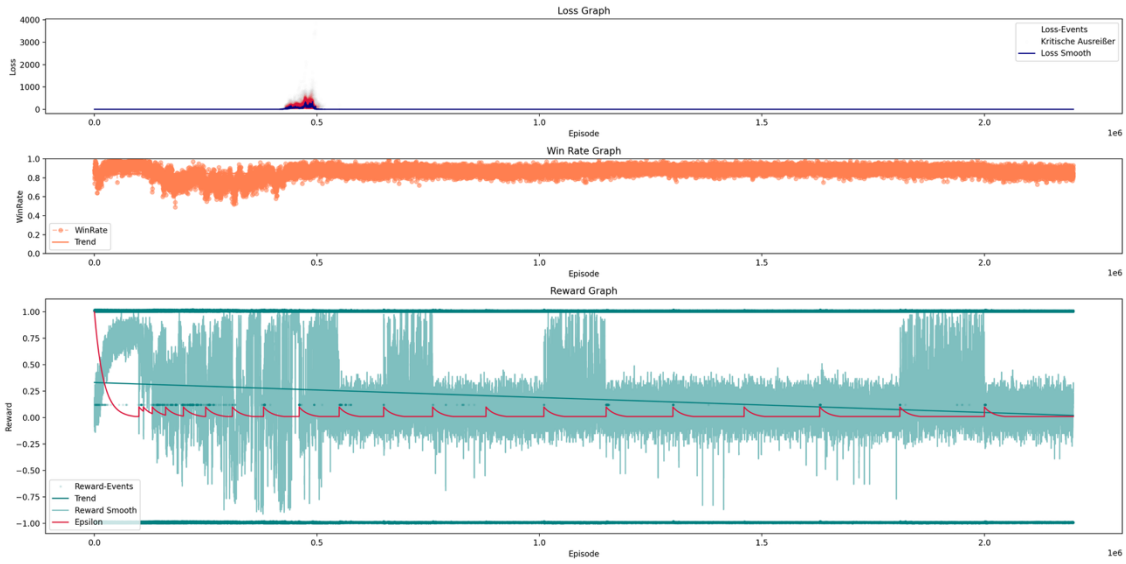

Obwohl hier die Hyperparameter identisch sind bis auf den Gamma-Wert, gibt es einen großen Unterschied in der Stabilität des Trainings. Das kann daran liegen, dass der niedrigere Gamma-Wert und die negative Überlebensbelohnung sich mehr oder weniger widersprechen.

Außerdem scheint es keine klare Korrelation zwischen den verschiedenen Metriken zu geben. Auch hier zeigt sich der Trend aus Trainer 3, dass Overfitting in den späteren Trainingsphasen auftritt, es gibt jedoch keinen Zusammenhang zwischen Loss und Strategie gegenüber einem Menschen. Der dynamischste Agent ist league_play_5, obwohl dieser die niedrigste Win-Rate im Training hat.

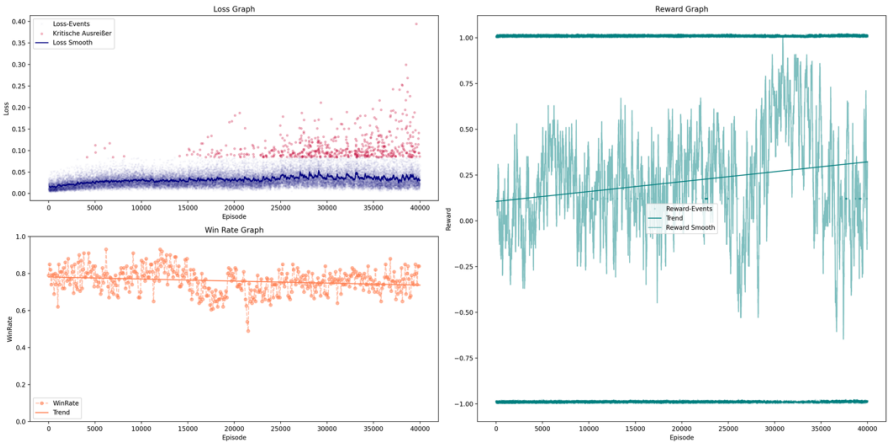
## 5.4 Vergleich zu Trainer 4

Die Hyperparameter von Trainer 7:

- Versteckte Schichtengröße: 256 Neuronen
- Gamma: 0,95
- Paketgröße: 128
- Überlebensbelohnung: +0,001



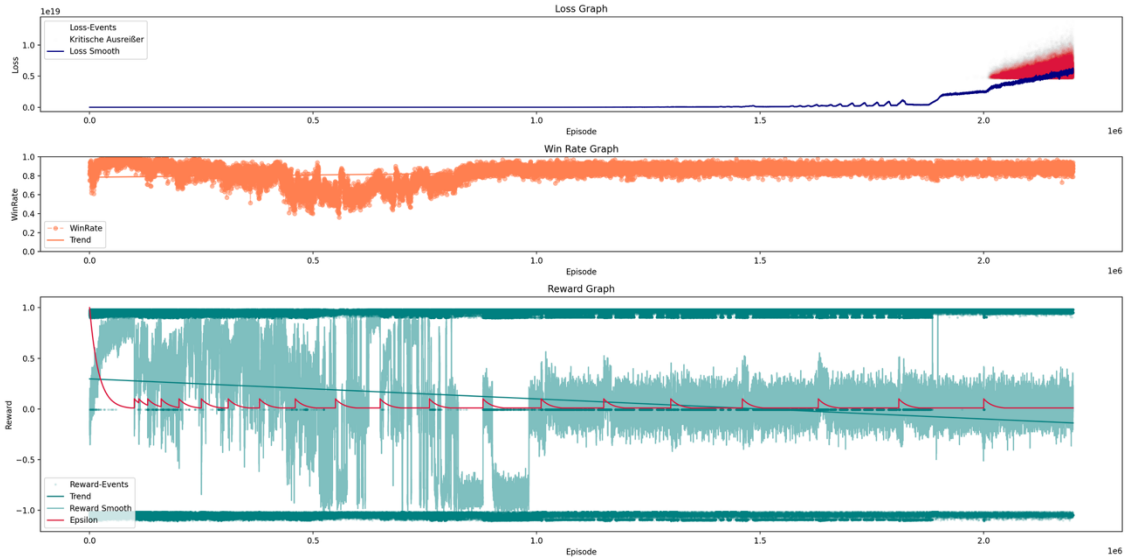

Die Daten vergleichen sich sehr gut mit Trainer 3, jedoch ist der Loss Ausbruch deutlich größer. Auf die Qualität der Modelle hat dies jedoch kaum einen Einfluss. Die späteren Modelle ähneln von ihrer Spielweise erneut den anderen. Bei den früheren Modellen fällt auf, dass erneut ziemlich dynamisch und von allen Modellen dieser Trainingscharge mit am defensivsten agiert wird. Dies liegt wahrscheinlich an der positiven Überlebensbelohnung. Auffällig ist vor allem League-Play-Model 4 (untere Grafik), welches als einziges mehrmals einen Horizontalen Block durchführte.



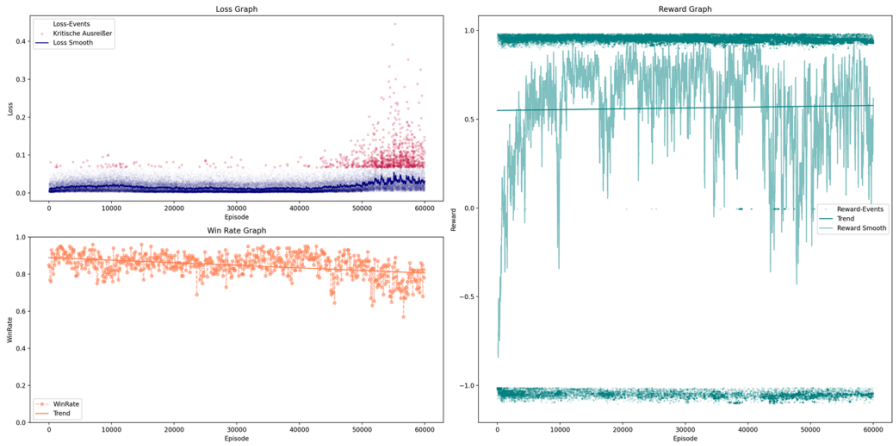
## 5.5 Vergleich zu Trainer 1

Die Hyperparameter von Trainer 1:

- Versteckte Schichtengröße: 256 Neuronen
- Gamma: 0,95
- Paketgröße: 64
- Überlebensbelohnung: -0,005



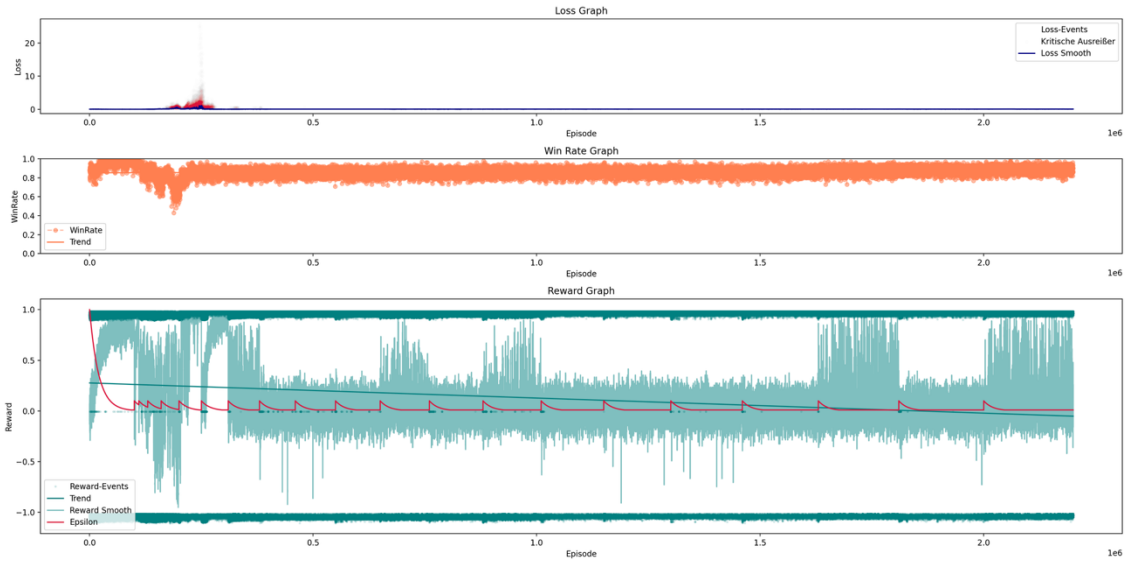

Die Daten von Trainer 1 weisen ein neues Muster auf. Gegen Ende des Trainings eskaliert der Loss-Wert und schießt bis weit über 10.000.000.000.000.000.000. Neben diesem Fakt ist auch auffällig, dass dies keinen negativen Einfluss auf die Win-Rate hat und die Belohnung an den Rändern „schmaler“ erscheint. Da die Überlebensbelohnung negativ ist heißt das, dass die Spiele im Durchschnitt deutlich kürzer dauern (sowohl Siege als auch Niederlagen). Self-Play-Model 6 (untere Grafik) scheint am kompetentesten zu sein, ohne erkennbaren Auffälligkeit in der Grafik.



## 5.6 Vergleich Trainer 11

Die Hyperparameter von Trainer 11:

- Versteckte Schichtengröße: 128 Neuronen
- Gamma: 0,95
- Paketgröße: 128
- Überlebensbelohnung: -0,005




Verglichen mit Trainer 3 sind die Hyperparameter identisch, jedoch ist das Netzwerk mit 128 Neuronen in den versteckten Schichten deutlich kleiner. Von den Daten her ähnelt es ebenfalls sehr stark Trainer 3 mit ähnlichen Ausreißern in allen Metriken. Die Agenten sind zum Ende hin weniger festgefahren auf eine Strategie als die anderen Modelle, jedoch wirken die Aktionen häufiger für einen Menschen willkürlich. Außerdem springt keines der Modelle als besonders kompetent oder auffällig heraus.

## 5.7 Zusammenfassung und Sonderheiten

Was im Trainingsprozess auffällt: Die Modelle sind bei weitem nicht unbesiegbar; im Gegenteil. Die meisten Agenten überpassen sich, vor allem zum Ende hin, an die Strategien im Agentenpool an und schaffen es nicht auf eine allgemeine Strategie zu generalisieren.

Einige weitere Fehler lassen sich an den getroffenen Designentscheidungen ableiten. Dass der Agent ein Vertikales Spiel bevorzugt, liegt einerseits an der eindimensionalen Übergabe des Spielfeldes, was ein klares geometrisches Verständnis klar erschwert, aber

andererseits auch die Entscheidung keine illegalen Züge zuzulassen. Dadurch, dass der Agent nie abspeichert, dass wenn bereits 6 Steine in einer Spalte liegen, ein weiterer nicht hinzukommen kann oder sollte, sondern die Q-Value für volle Spalten von außerhalb des Netzwerks überschrieben wird. Dadurch „versteht“ das Netzwerk selbst diese Regel in Kombination damit, dass es kein räumliches Verständnis hat, nicht, sondern kann lediglich nicht diese illegale Aktion machen.

Dadurch kommt es häufig vor, dass in einer Spalte z.B. bereits die 3 Reihen besetzt sind und auf diese trotzdem vertikal weiter aufstapelt, um vertikal zu gewinnen.

Das Overfitting kann im League-Play auch darin begründet liegen, dass bei langen League-Play-Sessions Epsilon zu schnell auf 0,01 sinkt und der Agent kaum noch erkundet.

# 6 Fazit

Die ausgiebigen Experimente haben klar zeigen können: ein DQN und vor allem ein DDQN können sehr effektiv Vier Gewinnt lernen; ABER wie groß der Erfolg ist, hängt stark von den für den Agenten gesetzten Rahmenbedingungen ab.

Die grundlegendste Grenze liegt in der Architektur selbst: Lineare Schichten besitzen kein räumliches Verständnis. Das Spielfeld wird als eindimensionaler Vektor übergeben, wodurch geometrische Zusammenhänge wie etwa horizontale oder diagonale Viererreihen für das Netz schwer zu erkennen sind. Dies erklärt die beobachtete Bevorzugung vertikaler Strategien. Ein konvolutionelles Netz (CNN), das das Spielfeld als zweidimensionale Matrix verarbeitet, würde hier strukturell bessere Voraussetzungen bieten.

Darüber hinaus tritt in allen Trainingsvarianten Overfitting auf: Die Agenten passen sich im League-Play zunehmend an die engen Verhaltensmuster des eigenen Agentenpools an, anstatt eine allgemeine Spielstrategie zu entwickeln. Dieses für Reinforcement Learning spezifische Overfitting ließe sich durch einen diverseren Agentenpool oder eine langsamere Epsilon-Senkung abmildern.

Die Untersuchung hat gezeigt, dass Deep Reinforcement Learning ein sehr effektiver Ansatz sein kann, gerade für komplexe Probleme, welche sich nicht einfach durch deterministische Abfragen darstellen lassen, doch für eine optimale Lösung des Problems Vier Gewinnt wäre eine CNN-basierte Architektur, ein diverserer Trainingsaufbau und eine umfangreichere Hyperparameter Anpassung notwendig.

# 7 Literaturverzeichnis

Die folgenden Quellen wurden in der Arbeit explizit referenziert:

*Volodymyr Mnih, Koray Kavukcuoglu, David Silver, Alex Graves, Ioannis Antonoglou, Daan Wierstra, Martin Riedmiller (2013): Playing Atari with Deep Reinforcement Learning, URL: https://arxiv.org/pdf/1312.5602 [Zuletzt abgerufen am 11.03.26]*

*Volodymyr Mnih, Koray Kavukcuoglu, David Silver, Alex Graves, Ioannis Antonoglou, Daan Wierstra, Martin Riedmiller (2015): Human-level control through deep reinforcement learning, in: Nature, Vol. 518, S. 529., https://storage.googleapis.com/deepmind-media/dqn/DQNNaturePaper.pdf [Zuletzt abgerufen am 11.03.26]*

*Richard S. Sutton, Andrew G. Barto (2018): Reinforcement Learning: An Introduction, 2. Aufl., URL: http://incompleteideas.net/book/RLbook2020.pdf [Zuletzt abgerufen am 11.03.26]*

*Hado van Hasselt, Arthur Guez, David Silver (2015): Deep Reinforcement Learning with Double Q-learning, URL: https://arxiv.org/pdf/1509.06461 [Zuletzt abgerufen am 11.03.26]*

*Weitere zur Vorbereitung genutzte, aber nicht referenzierte Quellen:*

*IBM (2026): The 2026 Guide to Machine Learning, IBM, URL: https://www.ibm.com/think/machine-learning#605511093 [Zuletzt abgerufen am 11.03.2026*

*PyTorch (2026): PyTorch documentation, URL: https://docs.pytorch.org/docs/stable/index.html [Zuletzt abgerufen am 13.03.2026]*

*YouTube (2026): Bellman Equation Basics for Reinforcement Learning, URL: https://www.youtube.com/watch?v=14BfO5lMiuk [Zuletzt abgerufen am 10.03.26]*